



DENDRITES

AI-Powered Learning Companion & Persistent Knowledge System

BROTOTYPE KOCHI • PROJECT APPROVAL

| What is Dendrites?

Solving "Context Amnesia"

At its core, Dendrites is a full-stack web application designed to move beyond disposable, single-session chats.

It provides a **structured, hierarchical workspace** where knowledge accumulates over time.

INTEGRATED WORKSPACE

Functions similarly to an **IDE** combined with a smart knowledge base.

PERSISTENT MEMORY

Moves beyond disposable chats to create an environment where **knowledge accumulates**.



What Does it Do?

1. Persistent Context Memory

Dendrites uses a 5-layer system to track your tech stack, learning goals, and past history.

- **Global Memory:** Cross-session profile & preferences.
- **Sliding Window:** Context of the last 20 messages.
- **Rolling Summaries:** AI-compressed long-term context.
- **Vector RAG** Semantic search through code and docs.
- **Spaced Repetition:** Flashcards via SM² algorithm.

2. Hierarchical Knowledge

Organize learning into deeply nested folders with **Context Inheritance** and Git-like **Branching**.

3. Document Ingestion

Upload PDFs, code, or images for semantic chunking and split-screen chat analysis.

4. "Viral" Knowledge Sharing

Generate shareable links for any folder. Users can "deep copy" entire trees with AI behaviors into their own account.



Who is it for?



Students & Learners

AI-Powered Learning Companion

- **Active Retention:** Designed to learn, retain, and build on conversations rather than disposable chats.
- **Spaced Repetition:** Create flashcards directly from AI responses for long-term memorization.
- **Learning Guardrails:** Ensures the AI stays strictly within a learning and education context.



Technical Users

Optimized for Technical Sessions

- **Stack Awareness:** Remembers personal tech stacks and experience levels across multiple sessions.
- **Code-Centric RAG:** Specialized vector databases for "code_blocks" and local models optimized for code.
- **Smart Workflows:** Ghost text autocomplete and full syntax highlighting support for complex coding tasks.



Five Layers working in Concert



Client Layer

React 18 □ Vite + TS. Clean architecture with DI Container and smart caching via React Query.



API Layer

Express + Node.js. 36 Use Cases decoupled from framework via application layer logic.



Async Pipeline

BullMQ □ Redis. 5 dedicated workers handling summary, embedding, and summary extraction.

| Frontend: Modular Clean Architecture

 UI Architecture Dashboard

- ✓ **React 18 & TypeScript:** Strict typing across all layers from domain to UI.
- ✓ **DI Container Pattern:** Decouples repositories from custom hooks and components.
- ✓ **Global State:** Redux Toolkit for auth and explorer; TanStack Query for server state.
- ✓ **Rich Rendering:** KaTeX for math and React Virtuoso for large chat lists, Plant UML, Mermaid markdowns .

Backend: Use-Case Driven Design



Domain Layer: 9 Pure TS interfaces (contracts) with zero framework dependencies.



Application Layer: 36+ granular use cases (Execute Pattern) for chat, recall, and workers.



Infrastructure Layer: Concrete Mongoose & Qdrant implementations—swappable DBs.



Stateless Services: Key rotation, SSE, Jina v2 embeddings, and Semantic Chunking.

The Core Differentiator

5□Layer AI Memory System

Persistent Knowledge Hierarchy 5 Layer Memory



2. CRUD Operations

The system handles all Create, Read, Update, and Delete operations for folders and chats, leveraging backend use cases and frontend custom hooks ([useFileItemMutations](#)).

Create (C)

- **Folders:** New folders at root or nested ([CreateFolder](#)).
- **Chats:** Created inside any folder, auto-linking ID ([CreateChat](#)).

Read (R)

The explorer retrieves the full user directory tree mapping parent-child relationships.

[GetFolders](#), [GetChats](#)

Update (U)

- **Renaming:** Quick rename via context menus ([UpdateFolder](#), [UpdateChat](#)).
- **Restructuring:** Move items via drag-and-drop for reorganization.

Delete (D)

Delete individual chats or trees. Backend manages cascading removals.

[DeleteFolder](#), [DeleteChat](#)

Advanced File Explorer

Dendrites features a robust, hierarchical file explorer designed to manage nested folders and chats seamlessly.



Sidebar & Modal Views

Accessible via a collapsible sidebar for quick access, or a full-screen modal overlay for deep organization.



Context Menus

Right-click functionality on any node allows users to quickly rename, move, delete, or branch chats.



Drag-and-Drop

Intuitive drag-and-drop organization for restructuring learning paths.



Document Browser

A centralized modal to manage all uploaded files and documents associated with specific chats.

| Diagram Generation & Rendering

The system is capable of rich visual outputs, moving beyond standard text.



Mermaid Integration

Generate complex architectural graphs, flowcharts, and sequence diagrams using Mermaid syntax, rendered directly in the interface.



Markdown & Math

Full support for GitHub-flavored markdown, callouts, and LaTeX math rendering via KaTeX for complex scientific equations.



Visual Mode (P5.js)

Dendrites supports transitioning to an interactive Visual Mode.

- Build interactive sketches and simulations on the fly.
- 900×600 sandboxed canvas.
- Layout-safe controls (Pause, Play, etc.).

Hierarchical Chat & Smart UX



Chat Branching

Replacing traditional flat structures with a linear branching system (Git-like).

- Spawn new child chats from specific messages.
- Context inheritance up to 15 levels deep.



Quick Chat

"Sticky Note" functionality for focused side-conversations.

- Trigger modals from any selected AI response text.
- Persistent storage as SubChat documents.



Inline Prediction

"Ghost Text" autocomplete predicting user input in real-time.

- Powered by Qdrant semantic search.
- Advanced string prefix matching.

Document Ingestion Pipeline



Multi-Format Support

Seamlessly upload and process a wide variety of data sources.

- PDF, TXT, MD, CSV, JSON
- Source Code Files
- Image Documents



Semantic Chunking

Intelligent algorithm for superior context preservation.

- Embedding-based detection
- Smart boundary recognition
- Meaningful document units



Split-View Rendering

Optimized interface for simultaneous analysis and chat.

- React-pdf & Prism syntax
- Draggable divider control
- Persistent chat accessibility

Behavioral Inheritance & Viral Loops



Folder-Level AI

Custom system prompts per folder with recursive nearest-ancestor resolution. Higher personalization than Claude Projects.



Shareable Link Engine

Viral growth loop: Deep-copy full folder trees and behavioral prompts into recipient accounts with one click.

| Smart UX & Safety Systems



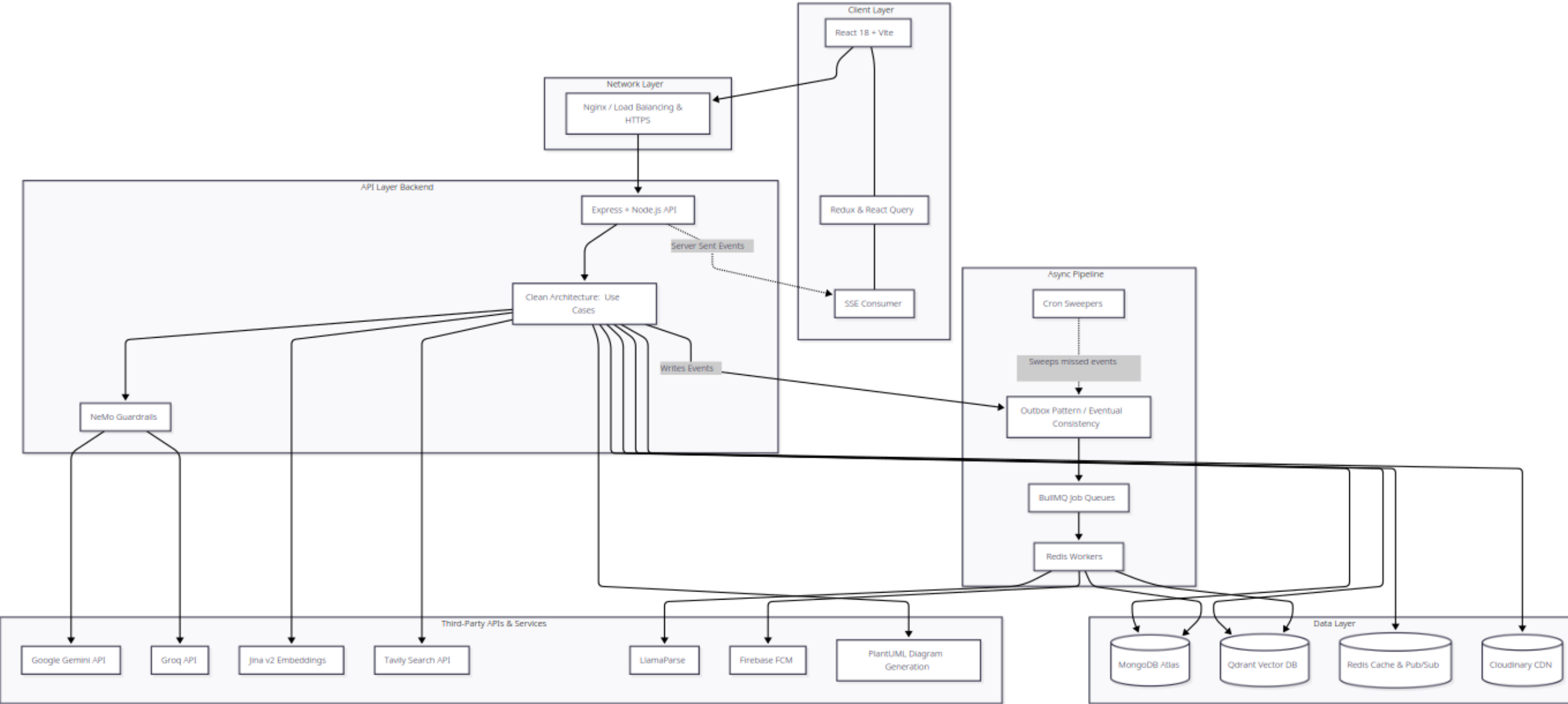
Inline Query Prediction

Ghost text autocomplete combining semantic similarity
[Qdrant] and past message prefix matching.



NVIDIA NeMo Guardrails

Safety layer blocking jailbreaks and off-topic requests
using open-source input/output rails via .co config.



Specialized Data Orchestration

Database	Role	Access Pattern
MongoDB Atlas	Primary Data Store	Document model for chat, folder, and memory schemas.
Qdrant Cloud	Vector Database	4 collections for RAG, summary, and semantic search cache.
Redis Cloud	Job Queue & Cache	BullMQ orchestration, Pub/Sub, and rate limiting.
Cloudinary	File CDN	Busboy streaming uploads for structurally parsed documents.

Production-Ready Security



Auth: JWT refresh/access token rotation via httpOnly cookies.



Isolation: userId-scoped queries enforced at both Mongo & Qdrant layers.



Files: Magic-byte type validation and streaming Busboy analysis.



Throttling: Plan-based AI quota enforcement middleware.



Questions?

Dendrites Architecture: A Full-Stack AI Learning Companion

